# Recommender Systems - Course Project Report

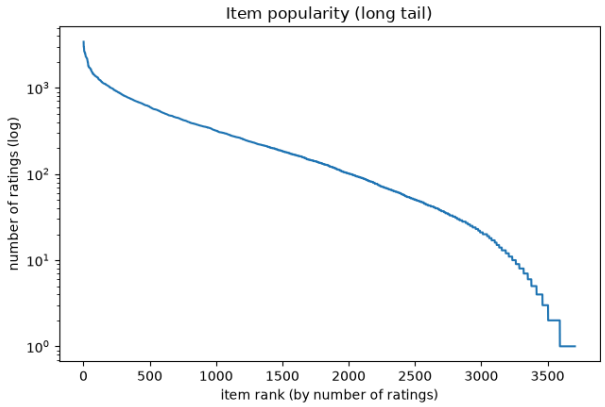
| | |
|---|---|
| **Team** | 16 |
| **Members** | • Jindřich Halabala (154807984)<br>• Pavel Halama (154808080)<br>• Amala Vijayan Pillai Soudha (153106068) |
| **For Part 3 track: select one** | A / B / C |
| **GitHub repository link** | https://github.com/halampav/DATA.ML.360-2026-2027-1_group16.git |

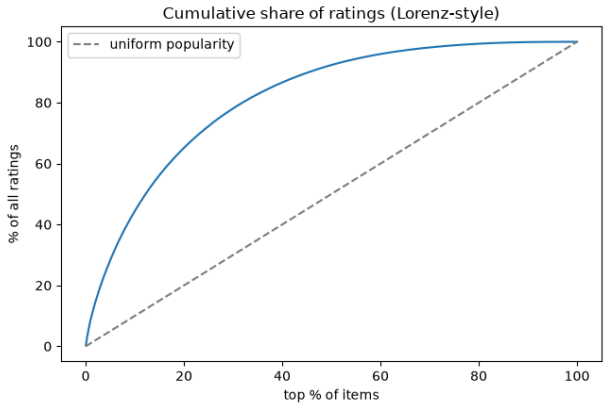
## 1. Part 1 - Data, Protocol and Baselines

### 1.1 Data characterization

MovieLens1M is a dataset that contains 6040 users (with some self-reported metadata) and 3883 movies (their name, year of release and genres) along with 1000209 ratings (each with a user and movie id, timestamp and a rating on a 1-5 scale). By design, every user has at least 20 ratings, but there are a total of 177 movies with no ratings. The density is 4.47 %.

The ratings are not distributed proportionally between movies or users. The median user made 96 ratings, but the maximum was 2314. The median movie has 123 ratings, while the most popular has 3428. There is a significant long tail where the top 10 % of most-rated movies accounts for 44.45 % of ratings and the top 20 % accounts for 65.16 %. 663 movies have fewer than 20 ratings.




The split between 1/2/3/4/5 star ratings is roughly the following (in percent): 6/11/26/35/23. Since 57.5 % of ratings are above or equal to 4, it seems like the ideal threshold for a movie to be considered relevant to the user. There are only two users with no ratings of 4 or above. The mean rating overall is 3.582.

Very importantly, the timestamps of the ratings are when they were entered, not when the user actually watched the movie. This means ratings for a single user are often not spread out over a longer time but instead bunched together in a small time period. 62.9 % of users entered all their ratings in a single day and the median activity is only 1.1 hours. This is very important to remember during the data splitting, more on that later.

### 1.2 Preprocessing and feedback treatment

We did not find any duplicate ratings or obvious mistakes that would need fixing. We decided to do no filtering or modifications of the data during loading because we do not want to lose any information that could be useful to later. That includes not filtering out users with no ratings (which can appear due to splitting). We do this since we believe it is more honest and fair. Yes, it makes models that can not deal

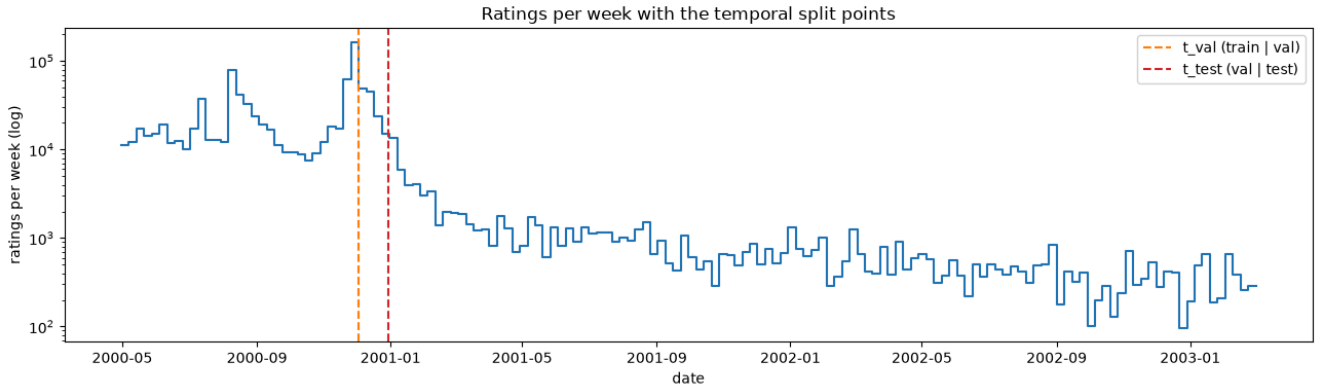
with cold starts have much worse scores, but that is sort of the point. That is a definite disadvantage of that model and it should be reported in the evaluation. At the same time, we do report both “warm only” and “cold users included” to make it clearer.

Some of the models do their own filtering. The 221 movies with no ratings in the training set are currently implicitly filtered by the baseline models because there is no way for them to recommend them. Our ItemKNN model also only recommends movies with at least 20 ratings (explained in section 1.5), which removes 765 movies (0.7 % of ratings).

As the relevance threshold, we chose any rating of >= 4 stars. This accounts for 57.5%, which is as close to a liked/disliked split as we can get. We checked whether this is actually fair since different users might have differing scales. Thus, we also calculated how much the relevant items would change if we defined them as above the user's mean. But the difference turned out to be small (around 13 %), while bringing some other problems, such as not being comparable to other teams who did not do this. Also, for users with few ratings, this mean could have a lot of variance, so in the end we went with the global threshold.

## 1.3 Train/validation/test split

As per the lecture recommendation, we used a temporal split on the 80 % (on 2000-12-2) and 90 % (on 2000-12-29) quantiles. This makes cheating due to “seeing the future” impossible and is the standard way of doing it. But with this dataset in particular, it does bring challenges. Some time periods are much more active than others. This can be seen in the figure below. Even though the training set contains 80 % of all ratings, it spans 220 days, while the 10 % validation and test sets last 27 and 790 days, respectively. This makes it difficult to use settings that rely on some absolute time period (such as the Most popular baseline model’s relevant day count).




Even more importantly since, as mentioned above, most users entered their ratings in a short time period the temporal split often splits different users into different sets rather than portions of a user’s ratings. This leads to a lot of cold-start users (no ratings in training data). 54 % of users in the validation data have no ratings in the training data. This drops to just 16.5 % for the test set (and to just 2.4 % when the model is trained on train+val). As mentioned above, we decided not to filter them out since we think it is fairer. But it is very important to remember during the evaluation, especially the big difference between validation and test sets, which causes a big difference in results for models that can not handle cold-start users.

## 1.4 Evaluation protocol

For evaluation, we use the whole catalogue of movies (except the ones the user has already rated in the training data), as recommended by lecture 3.1 slide 5, since using just a part can lead to biased results. This includes movies that were never rated (and thus the baseline models will never recommend them, which lowers their coverage).

We evaluate every user who has at least one relevant movie (meaning at least one rating outside of the training data that is >= 4 stars). This includes cold-start users. Models, which can not handle cold users, do not recommend anything and are thus given a rating of 0 for such users.

We measured Precision@10 (note: many users do not have 10 relevant movies, so the best possible score is 0.82), Recall@10 and NDCG@10 averaged over all users. We also measure coverage (which percentage of movies was recommended at least once). We also measured these metrics for a random recommender and a perfect recommender to give us some bounds we could compare to. We implemented our own metrics and show that they match the ones from scikit-learn (see results/metric_check.csv).

All tuning was done on the validation dataset. The final training was done on the train+val dataset. Test data was only used once to measure the final metrics. All models are deterministic and don't use a seed, but we did use one for the tuning and the random baseline.

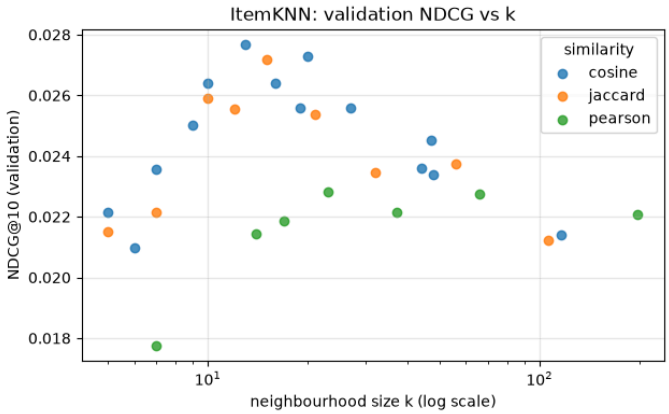
## 1.5 Baseline methods and settings

The item-based kNN model looks at what other movies that user rated, compares their similarity to the one we want to rate (similarity of two movies is calculated by comparing their rating vector) and uses those similarities as weights for a weighted average of the user's ratings to get the new rating (slightly simplified for the report).

We have the following standard parameters: k (how many most similar movies are used in the weighted average), similarity measure (cosine, Pearson, Jaccard) and gamma (significance weighting. Items with fewer co-ratings are treated as less similar up to a count gamma). This performed terribly for movies with few ratings. That is because their average rating had a lot of variance, so it was possible for such a movie and thus their ranking was often predicted as very high (or very low). This is not a problem for prediction, but in ranking, only just 10 such movies could completely push out all the actually good movies with more ratings. We thus added a new parameter, min_support. Only movies with at least this many ratings can ever get recommended

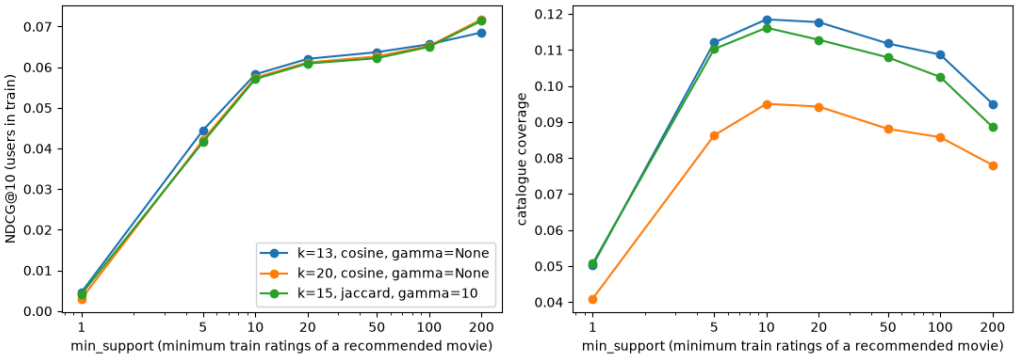
MostPopular is much simpler. It just recommends movies that have the most ratings. The parameters are how many of the last days we are taking into account and whether to only count positive reviews (and what is the threshold for a rating to be positive).

Since MostPopular is very simple and fast, we just tried all possibilities (only_positive: true/false, window_days: all time/90/30/7). The window made very little difference (and, as mentioned, tuning it on validation data is not the best since the last x days may contain very different amounts of data once evaluating on the test set). Counting only positive reviews raised all the metrics by around 0.01 (except coverage, which did not show a clear trend).




For ItemKNN, we used the Optuna framework (TPE sampler) for hyperparameter tuning mainly because it has more parameters and is way slower. We ran 30 trials for just k, similarity measure and gamma, which ran for around 3 hours. Gamma seemed to have little effect, but k had a nice where it was the best at around 10 to 20. Cosine and Jaccard scored similarly, while Pearson was consistently worse (and also slower).

We then tuned min_support separately on just the three best configurations from before. All three got a lot better with higher min_support. But as is also visible from the figure below, the coverage started decreasing noticeably after 20 while the NDCG@10 was rising only slightly. For that reason, we decided to go with k=13, cosine, gamma=None and min_support=20, as it is the best compromise.




## 1.6 Results and analysis

| Model | P@10 | R@10 | NDCG@10 | Coverage |
|---|---|---|---|---|
| Perfect ranker | 0.837 | 0.529 | 1 | 0.410 |
| Random ranker | 0.013 | 0.003 | 0.012 | 0.957 |
| Most popular, positive only, 7 last days | 0.196 | 0.067 | 0.216 | 0.032 |
| ItemKNN (k=13, cosine, min_support=20, no gamma) | 0.090 | 0.031 | 0.096 | 0.160 |

Our final results show that the very simple MostPopular model actually outperforms the more complex tuned ItemKNN by a significant margin on all accuracy-based metrics (coverage is logically worse for MostPopular). Notably, MostPopular performs way better on cold users than warm users (probably because new users are more likely to rate already popular movies), while ItemKNN does not work on cold users at all (this makes a big difference on validation data where over half of users are cold and a smaller difference on the test data. That is why the differences between these two are so big). The parameters made little difference for MostPopular but a pretty big difference for ItemKNN (especially min_support).

# 5. Contribution Statement

Pavel implemented the data loading, splitting and performed the dataset analysis. Jindřich implemented the baseline models and tuned them. Amala wrote the functions for calculating metrics and did the evaluation. We all collaborated on the report.

# 6. Use of AI Assistants

Claude Code was used to help write the notebooks for data analysis and tuning, though their content was heavily curated by us. The initial implementations of the baseline models were written by hand but were then rewritten by Claude to use pandas and vectorization more effectively, as the handwritten ones were super slow.